

Mejoras de rendimiento — academia.conaf.cl

Estado: borrador para discusión. No se ha aplicado nada. Fecha: 29 de julio de 2026 · Autor técnico: Luis Monsalve

Contexto

El sitio quedó funcionando sobre PostgreSQL 17 tras la migración, pero se percibe **lento en todo el sitio**: portada, inicio de sesión, "Mis cursos" y al entrar a los cursos. En cambio **no** se percibe lentitud al abrir o descargar archivos.

Ese patrón es información: si abrir un PDF de varios MB va bien pero cargar una página va mal, el cuello de botella **no** es el bind mount de los 10,9 GB ni el disco. Apunta a configuración de Moodle, a la base de datos, o a la caché.

Nada de esto se ha medido todavía. Es una percepción al navegar: un punto de partida legítimo pero insuficiente, porque sin números no se puede saber si un cambio mejoró algo. Por eso el plan empieza midiendo.

La regla que ordena todo el documento

Medir, cambiar una cosa, medir de nuevo.

Aplicar seis mejoras a la vez y ver que "va más rápido" no enseña nada: no se sabe cuál sirvió, ni si alguna empeoró algo, ni qué mantener si hay que revertir.

El argumento para medir **no** es ahorrar reconstrucciones de la imagen —eso es barato y el despliegue automático ya está probado—. Es que con **langstringcache** en 0 es perfectamente posible que un solo cambio de un comando explique casi toda la lentitud, y en ese caso las fases 4 en adelante dejan de ser necesarias.

Evidencia ya verificada

Comprobado leyendo el volcado y el código de Moodle 4.5.10. No son suposiciones.

Configuración de Moodle heredada del sitio original

Ajuste	Valor actual	Debería ser	Por qué importa
langstringcache	0	1	Con 0, Moodle vuelve a leer y parsear los archivos de idioma en cada petición . El paquete es instalado tiene 1.570 archivos . Es el sospechoso número uno

Ajuste	Valor actual	Debería ser	Por qué importa
<code>debug</code>	15 (NORMAL)	0	Recolecta información de depuración en cada página, aunque <code>debugdisplay=0</code> la oculte
<code>perfdebug</code>	7	0 (después de medir)	Instrumentación de rendimiento activa de forma permanente
<code>themedesignermode</code>	0	0	Correcto. Es la causa clásica de lentitud extrema en Moodle, y aquí está bien
<code>cachejs</code>	1	1	Correcto
<code>cachetemplates</code>	1	1	Correcto
<code>enablestats</code>	0	0	Correcto

Infraestructura

Hecho	Estado
Caché de aplicación de Moodle (MUC)	Archivos , en <code>moodledata/muc</code> y <code>cache/</code> → sobre el bind mount
Sesiones	Archivos , en <code>moodledata/sessions</code> → sobre el bind mount
Redis / Memcached	No hay ninguno
<code>cache/stores/redis</code>	Viene en el core de Moodle 4.5: no hace falta instalar plugin
Extensión <code>redis</code> de PHP	NO está en la imagen (solo gd, intl, zip, soap, exif, opcache, mysqli, pgsql)
PostgreSQL	En otra máquina (172.31.2.40), compartida con otras apps de CONAF. Cada consulta cruza la red
Estadísticas del planificador	Nunca se ejecutó <code>VACUUM ANALYZE</code> después de que <code>dbtransfer</code> insertara ~1,5 millones de filas
<code>mariadb-tmp</code>	Sigue encendido , con <code>innodb_buffer_pool_size=2G</code> reservados
Archivos <code>.php</code> en la imagen	18.254 · <code>opcache.max_accelerated_files=20000</code> : alcanza, con poco margen
Nginx del host	Sin gzip y sin caché de estáticos
PHP	<code>mod_php</code> con MPM prefork (un proceso completo por petición)

Fase 0 — Medir. No cambia nada

0.1 Separar la red del servidor

```
# En el servidor: tiempo real de Moodle, sin la red del cliente de por medio
for i in 1 2 3 4 5; do
    curl -s -o /dev/null -w '%{time_total}s http=%{http_code}\n' \
        -H "Host: academia.conaf.cl" http://127.0.0.1/login/index.php
done
```

Y lo mismo desde una estación de trabajo contra <http://academia.conaf.cl>. Si el tiempo en el servidor es bajo y desde el equipo es alto, **el problema es la red o la VPN, no el sitio**, y el resto de este documento no aplica.

0.2 Ver dónde se va el tiempo dentro de Moodle

```
docker compose exec -u www-data app php admin/cli/cfg.php --name=perfdebug --
set=15
docker compose exec -u www-data app php admin/cli/purge_caches.php
```

Con `perfdebug=15`, Moodle muestra al pie de cada página el tiempo de generación, la memoria usada y **el número de consultas a la base de datos**. Hay que navegar la portada, "Mis cursos" y un curso, y anotar los tres números de cada una.

Eso es lo que convierte "está lento" en un diagnóstico:

- **Muchas consultas (cientos) y mucho tiempo** → el cuello es la base de datos remota.
- **Pocas consultas pero mucho tiempo** → el cuello es PHP: cadenas de idioma, caché, plantillas.
- **Mucha memoria** → algo está cargando más de lo que debería.

Dejarlo en 0 al terminar.

0.3 Recursos de la máquina

```
free -h ; nproc ; uptime
docker stats --no-stream
```

`docker stats` va a mostrar cuánta memoria se está llevando `mariadb-tmp`, que ya no hace falta encendido.

0.4 Estado del cron

```
docker compose logs --tail=30 cron
docker compose exec -u www-data app php admin/cli/checks.php
```

Ya apareció una notificación de `tool_task_maxfaildelay`, que indica tareas programadas fallando. Si el cron no corre bien, hay páginas que calculan al vuelo lo que debería estar precalculado.

Salida de esta fase: una tabla con tiempo, consultas y memoria para tres páginas.

Fase 1 — Ajustes de configuración de Moodle

Reversibles, inmediatos, sin reconstruir la imagen ni reiniciar contenedores.

Orden	Cambio	Riesgo
1	<code>langstringcache</code> de 0 a 1	Ninguno: es el valor por defecto de Moodle
2	<code>debug</code> de 15 a 0	Ninguno. Se sube temporalmente cuando haya que diagnosticar
3	<code>perfdebug</code> a 0 , al terminar la Fase 0	Ninguno

```
docker compose exec -u www-data app php admin/cli/cfg.php --name=langstringcache -
-set=1
docker compose exec -u www-data app php admin/cli/cfg.php --name=debug --set=0
docker compose exec -u www-data app php admin/cli/purge_caches.php
```

Medir de nuevo 0.1 y 0.2 después de este solo cambio. Si `langstringcache` era la causa, la diferencia va a ser evidente y el resto del documento pasa a ser opcional.

Fase 2 — Estadísticas de PostgreSQL

El otro candidato principal, y el más fácil de pasar por alto.

`dbtransfer` insertó ~1,5 millones de filas, fila por fila, a través de PHP. PostgreSQL decide cómo ejecutar cada consulta a partir de estadísticas sobre la distribución de los datos, y **esas estadísticas no existen hasta que alguien ejecuta ANALYZE**. Sin ellas el planificador trabaja con estimaciones por defecto y elige recorridos secuenciales de tabla completa donde debería usar un índice.

Con 178.833 filas en `mdl_files` y la base al otro lado de la red, eso se paga en cada página.

```
PGPW=$(grep '^DATABASE_PASSWORD=' /opt/apps/coipo_moodle/.env | cut -d= -f2-)
docker run --rm -e PGPASSWORD="$PGPW" -e PGGSSENCMODE=disable -e PGSSLMODE=disable \
postgres:17-alpine psql -h 172.31.2.40 -U academia -d academia_prod \
-c "VACUUM (ANALYZE, VERBOSE);"
```

- Se ejecuta como el rol `academia`, dueño de la base: **no toca las otras apps** del PostgreSQL compartido.
- No bloquea el sitio: `VACUUM` sin `FULL` es concurrente.
- Toma unos minutos.

Para comprobar antes y después si las estadísticas existían:

```
SELECT relname, n_live_tup, last_analyze, last_autoanalyze
FROM pg_stat_user_tables
WHERE relname IN
('mdl_files', 'mdl_user', 'mdl_course_modules', 'mdl_context', 'mdl_role_assignments')
ORDER BY relname;
```

Si `last_analyze` y `last_autoanalyze` salen vacíos, queda confirmado que el planificador estaba trabajando a ciegas.

Medir otra vez después de esto.

Fase 3 — Liberar la memoria del MariaDB temporal

`mariadb-tmp` sigue levantado con 2 GB reservados para su buffer pool, compitiendo por la memoria de la máquina con PHP y con las cachés. Ya no hace falta encendido: su función terminó cuando `dbtransfer` copió los datos.

```
docker stop coipo_moodle-mariadb-tmp-1
free -h
```

El volumen `coipo_moodle_mariadb_tmp` NO se borra. Es el rollback: si algún día hay que volver atrás, se levanta el contenedor otra vez y los datos siguen ahí. Solo se apaga el proceso.

Fase 4 — Un solo rebuild: Redis, extensión y margen de OPcache

La mejora estructural de fondo. Requiere reconstruir la imagen, pero eso no es un obstáculo: el despliegue automático está probado y un push a `main` reconstruye y redespiega solo. Por eso **conviene juntar en un mismo rebuild todo lo que toca la imagen** en vez de repartirlo en varios.

Cambio	Archivo	Por qué
Extensión <code>redis</code> de PHP	<code>Dockerfile</code>	Sin ella Moodle no puede hablar con Redis
<code>opcache.max_accelerated_files</code> 20000 → 24000	<code>docker/php.ini</code>	Hay 18.254 archivos <code>.php</code> . Alcanza, pero con poco margen. Gratis y sin riesgo
<code>opcache.interned_strings_buffer</code> 16 → 32	<code>docker/php.ini</code>	Moodle usa muchísimas cadenas repetidas
Sesiones a Redis	<code>docker/config.php</code>	Ver abajo

Qué resuelve

Hoy la caché de aplicación de Moodle y las sesiones son **archivos sobre el bind mount**. Con un usuario apenas se nota; con cientos concurrentes, cada petición pelea por bloqueos de archivo en un sistema de archivos montado. Las sesiones de archivo en Moodle son un cuello de botella conocido: Moodle bloquea el archivo de sesión durante **toda** la petición.

Redis lo mueve a memoria, dentro de la red interna de Docker.

Qué implica

1. **Extensión redis de PHP**: `pecl install redis && docker-php-ext-enable redis` en el `Dockerfile`. Si `pecl` falla por falta de compilador hay que agregar `$PHPIZE_DEPS` antes —la imagen oficial de PHP lo documenta para este caso—, aunque probablemente no haga falta porque el `Dockerfile` ya compila `gd` e `intl` sin problemas.
2. **Servicio redis en docker-compose.yml**, siguiendo el patrón CONAF: **sin puerto publicado**, alcanzable solo por `app` y `cron` por la red interna de Docker. Con `maxmemory` y política `allkeys-lru` para que no crezca sin límite.
3. **Sesiones a Redis** en `docker/config.php`, con los ajustes verificados en el `config-dist.php` del propio Moodle:

```
$CFG->session_handler_class = '\core\session\redis';  
$CFG->session_redis_host = 'redis';  
$CFG->session_redis_port = 6379;  
$CFG->session_redis_acquire_lock_timeout = 120;  
$CFG->session_redis_lock_expire = 7200;
```

4. **Caché de aplicación a Redis** desde la interfaz: Administración del sitio → Plugins → Caching → Configuration, moviendo el almacén de **Application**.

Qué NO mover a Redis

- `$CFG->localcachedir` se queda en el volumen de Docker. Está diseñado para ser local a cada nodo y desechable; moverlo a Redis no aporta nada.
- La caché **Session** de la MUC no es lo mismo que las sesiones de PHP. Cambiar las dos a la vez complica el diagnóstico si algo sale mal.

Riesgo

Un servicio más que operar. Y si Redis se cae, **se pierden las sesiones y todos quedan desconectados**. Mitigable con `restart: unless-stopped` y la política de memoria bien puesta. El corte sería de sesión, no de datos: nada de la base depende de Redis.

Fase 5 — Nginx

Fuera del repositorio: es el vhost del host, `/etc/nginx/sites-available/academia.conaf.cl.conf`.

Falta gzip. Moodle sirve CSS y JavaScript grandes; sin compresión viajan enteros en cada carga en frío. Es el cambio más barato de todo el documento:

```
gzip on;
gzip_vary on;
gzip_min_length 1024;
gzip_types text/plain text/css text/javascript application/javascript
application/json image/svg+xml;
```

Moodle ya manda cabeceras de caché largas para los recursos versionados por `themerev` y `jsrev`, así que **no hay que agregar caché de estáticos en Nginx**: duplicaría esa lógica y complicaría los despliegues de tema.

Fase 6 — Solo si sigue lento después de todo lo anterior

Opción	Cuándo tiene sentido	Riesgo real
php-fpm en vez de mod_php	Si <code>docker stats</code> muestra la memoria al límite con pocos usuarios concurrentes: prefork levanta un proceso completo por petición y cada uno carga PHP entero	El coste no es reconstruir, es que cambia el comportamiento : hay que rehacer la configuración de Apache o pasar a nginx dentro del contenedor, y se toca lo único que hoy funciona con certeza
Retención de logstore_standard	Si esa tabla crece mucho y las consultas de informes se degradan	Cambiar la retención, nunca borrar la traza de auditoría : en un servicio público hay que conservarla
\$CFG->dbpersist	Con la base en otra máquina, ahorra establecer la conexión en cada petición	Con <code>mod_php</code> prefork y muchos procesos puede agotar el <code>CONNECTION LIMIT 60</code> del rol y dejar el sitio sin conexiones. No activar a ciegas : medir primero cuántas conexiones se usan
Índices adicionales	Solo si una consulta concreta aparece lenta y medida. <code>dbtransfer</code> creó todos los índices del <code>install.xml</code>	Hay que identificar la consulta primero. Un índice de más también cuesta en cada escritura

Qué NO hacer

- **No reescribir `mdl_logstore_standard_log`**. Es traza de auditoría y `tool_replace` la salta por diseño (ver `docs/MIGRACION.md`).
- **No tocar la configuración del PostgreSQL compartido** (`shared_buffers`, `work_mem`) sin hablar con quien administra 172.31.2.40: esa máquina sirve a otras apps de CONAF.
- **No poner `opcache.validate_timestamps=0`** para ganar unos milisegundos. La imagen se reconstruye en cada deploy y el riesgo de servir código viejo no lo compensa.
- **No desactivar el registro de eventos** para ganar velocidad: es un requisito de auditoría.
- **No aplicar varias fases a la vez**. Sin medición intermedia no se sabe qué funcionó.
- **No borrar todavía** el volumen de MariaDB ni el volcado: son el rollback de la migración.

Verificación

Una sola tabla, llenada con los comandos de la Fase 0 después de cada fase:

Medición	Base	Tras F1	Tras F2	Tras F3	Tras F4
<code>time_total</code> de <code>/login/index.php</code> en el servidor					
Tiempo de la portada (pie de Moodle)					
Consultas a la base en la portada					
Tiempo de "Mis cursos"					
Consultas en "Mis cursos"					
Memoria de un curso					
<code>free -h</code> disponible					

Si una fase no mueve ningún número, se anota y se descarta. Eso también es información.

Resumen para la discusión

1. Hay **dos sospechosos principales**, ambos verificados y ambos gratis de arreglar: `langstringcache=0` —Moodle reparseando 1.570 archivos de idioma en cada petición— y la falta de `VACUUM ANALYZE` tras insertar 1,5 millones de filas.
2. La causa **no** parece ser el bind mount de los 10,9 GB, porque abrir archivos va bien.
3. **Reconstruir la imagen no es un problema.** Conviene juntar en un solo rebuild la extensión de Redis y el margen de OPcache. Lo que sí hay que sopesar de Redis no es el rebuild: es que **es un servicio más que operar.**
4. Falta gzip en Nginx. Es gratis y está fuera del repositorio.
5. **Nada de esto se ha medido.** Las fases 0 a 3 se hacen en una tarde, y puede que con eso baste.